

A Split-Second Forgery: A Zero-Query Attack on Random-Split St-Gen

Martin Feussner

Selmer Center, University of Bergen

`martin.feussner@uib.no`

9 October 2026

AI disclosure and review status. OpenAI Codex generated and selected this paper’s title and intentionally chose “A Split-Second Forgery” as wordplay on the scheme’s random splitting and the millisecond attack. OpenAI Codex performed the scheme reconstruction, proof development, implementation, experiments, and technical writing, with specialist AI agents used for primary, independent, history, scope, adversarial, and publication review. The primary reproduction, independent reproduction, exact-history, intended-scope, amended adversarial, and independent publication-review gates all passed. The result is classified **VERIFIED_ATTACK**. No independent human verification was performed or required.

Abstract

We describe a public-key-only, zero-query existential forgery against the literal raw-message interface in Algorithms 5–6 of Gligoroski and Samardjiska’s random-split St-Gen signature proposal, IACR ePrint 2016/391. The signed value specified by the paper is a tuple $z = (z_1, z_2) \in (\mathbb{F}_2^n)^2$. Verification computes $e'_i = \sigma G_{i,\text{pub}} + z_i$ and accepts when every block pair belongs to the public **ValidErrorSplits** relation. An attacker chooses any $\sigma \in \mathbb{F}_2^k$ and public blockwise-valid e_1, e_2 , then sets $z_i = \sigma G_{i,\text{pub}} + e_i$. Characteristic-two cancellation makes the verifier recover exactly the selected errors. The result is a fresh attacker-chosen raw- z forgery with no secret input and no signing query.

Two independently structured standard-library implementations reproduce the identity at both printed full rows: $(n, k, \ell, s) = (2142, 465, 3, 2)$ and $(2244, 508, 4, 2)$. The sealed primary run accepted zero- and nonzero- σ variants for each row, covering all 2,550 positive blocks. Its slowest attack-only measurement was 1.867 milliseconds; the full run, including key generation and controls, took 3.21 seconds and 20,992 KiB peak resident memory. Invalid-block, changed-signature, and representation controls reject. Our claim is confined to the raw z interface printed in Algorithms 5–6. We make no claim for arbitrary documents or an unspecified external document-to- z hash transform, and no novelty claim for verification-first existential forgery or linear cancellation. The exact-history, intended-scope, amended adversarial, and independent publication reviews passed, and the result is classified **VERIFIED_ATTACK**.

1 Introduction

Gligoroski and Samardjiska propose a code-based signature scheme whose public key consists of two randomly split and independently permuted generator matrices [1]. The split is designed to frustrate structural and information-set-decoding attacks considered for earlier St-Gen schemes. The signature verifier, however, exposes a simpler issue at the level of its input/output relation.

Algorithm 5 receives a raw tuple $z = (z_1, z_2)$ “to be signed.” Algorithm 6 receives the same tuple and a signature vector σ . Its only algebraic operation is to recover two error vectors by adding public codewords to the two message components. Because the relation checked afterward is public, an attacker can choose the checked errors first and solve the displayed verification equations for z . No inversion or decoding problem remains.

The attack has the following properties:

- it applies to the exact pinned ePrint target and both complete parameter rows printed in Section 4;
- it invokes the ordinary Algorithm 6 verifier and derives the complete public valid-split sets from Algorithm 3;

- it uses no private value and makes zero signing queries;
- it succeeds deterministically for every public key; and
- it forges a fresh attacker-chosen raw z tuple, the value that the paper itself supplies to Algorithms 5–6.

Classical technique attribution. Choosing a candidate signature first and using a public verification relation to obtain some matching message is a classical route to existential forgery. The standard hierarchy defines existential forgery as producing at least one accepted message/signature pair [6, §11.2.4]; the modern adaptive chosen-message security formulation was established by Goldwasser, Micali, and Rivest [5]. We make no novelty claim for verification-first existential forgery, linear cancellation, or the general technique. Our contribution is its concrete application to the intended raw- z verifier in ePrint 2016/391. No prior public attack against the random-split St-Gen signature scheme was located in the searches performed as of 2026-10-09. This is a dated search result, not an absolute priority claim. Moody and Perlner’s earlier practical cryptanalysis applies to the unsplit predecessor and is credited separately [4].

2 Target and claim boundary

2.1 Pinned target

The target is *A Digital Signature Scheme Based on Random Split of St-Gen Codes* by Danilo Gligoroski and Simona Samardjiska, IACR ePrint 2016/391 [1]. The exact PDF used by the audit has SHA-256 digest

b2bdf97bcef5dfec9928749d0d3fad0a97956494fc81989c52cd4472dec5755e.

The extracted text has SHA-256 digest

69fafe4ab6f7bb2fad7dfd6687f66dd19c5d86e738163ac5bdf75cb7fbd06d6f.

The reconstruction is bound to this target and to the audit’s sealed primary and independent evidence.

2.2 Intended message space

The unsplit predecessor, *McEliece in the world of Escher*, explicitly describes its signing input as **Syndrome** = **Hash**(Doc) and contrasts that operation with the CFS loop **Hash**(Doc, Counter) [2]. Its later formal algorithms use a raw vector z . The predecessor therefore proves that this lineage knew how to state document hashing when intended. It does not, by itself, make that transform a mandatory unstated part of the 2016 target.

Target-specific implementation evidence resolves the remaining ambiguity. Samy Saad Samy Shehata’s 2017 NTNU master’s thesis, *Post Quantum Cryptography with random split of St-Gen codes*, was supervised by Danilo Gligoroski, a coauthor of the target [3]. The thesis states that it provides practical C implementations using the exact concrete parameter sets from the random-split encryption and signature papers. Its Algorithms 19 and 20 call $z_1, \dots, z_s$ the messages, place those raw binary vectors at the complete Sign/Verify boundary, and verify through

$$e_i = \sigma G_{i,\text{pub}} + z_i.$$

The appended interface passes z as a byte-array argument to the signature and verification routines, and the experimental correctness criterion is signing a binary vector followed by successful verification. No document-to- z transform is inserted at this interface.

The thesis is corroborating scope evidence rather than a replacement target. It is implementation-oriented, exact-parameter work under a target author’s supervision, and it shows that the raw-vector boundary was used deliberately after publication. The independent scope arbitration therefore treats the raw binary-vector space as the intended mathematical signature interface. A preimage-resistant external wrapper cannot be imported from the predecessor as a mandatory hidden step.

2.3 Public error-split relation

Let $E_\ell \subseteq \mathbb{F}_2^\ell$ be the error set and let $s = 2$ be the number of matrix splits. Definition 3 calls $(a, b) \in (\mathbb{F}_2^\ell)^2$ a valid error split when

$$\pi_1(a) + \pi_2(b) \in E_\ell$$

for every pair of coordinate permutations π_1, π_2 . Algorithm 3 enumerates all such pairs once, and the paper says the resulting `ValidErrorSplits` set may be public. Write that set as $\mathcal{V}$.

The two printed rows use error sets consisting of all ℓ -bit words of Hamming weight one or two. Exhaustive implementation of Algorithm 3 gives the counts stated by the paper:

Row	n	k	ℓ	blocks n/ℓ	$ \mathcal{V} $
PS1	2142	465	3	714	24
PS2	2244	508	4	561	40

2.4 Algorithms 5 and 6

For $s = 2$, Algorithm 5 takes

$$z = (z_1, z_2) \in (\mathbb{F}_2^n)^2$$

as the value to be signed and returns $\sigma \in \mathbb{F}_2^k$. It intends the resulting errors

$$e_i = \sigma G_{i,\text{pub}} + z_i$$

to have every corresponding block pair in $\mathcal{V}$.

Algorithm 6 receives (z, σ) and the two public matrices. It computes

$$e'_i = \sigma G_{i,\text{pub}} + z_i, \quad i \in \{1, 2\}, \quad (1)$$

and accepts precisely when every ℓ -bit block pair of (e'_1, e'_2) belongs to $\mathcal{V}$.

The target does not define a document-to- z encoding, a preimage-resistant hash-to- z transform, or a byte-level external-message format. Our result is therefore limited to the intended raw tuple accepted by the printed algorithms and corroborated by the thesis. We do not claim a forgery for arbitrary documents, a preimage under a separately defined wrapper, the ePrint 2014/360 predecessor, or any software implementation.

3 Zero-query forgery

Theorem 1. *For every public key generated by Algorithm 2, every $\sigma \in \mathbb{F}_2^k$, and every pair of vectors (e_1, e_2) whose corresponding ℓ -bit blocks all lie in $\mathcal{V}$, the tuple*

$$z_i = \sigma G_{i,\text{pub}} + e_i, \quad i \in \{1, 2\}, \quad (2)$$

is accepted with signature σ by Algorithm 6.

Proof. Substituting Equation (2) into the verifier's Equation (1) gives

$$e'_i = \sigma G_{i,\text{pub}} + \sigma G_{i,\text{pub}} + e_i = e_i,$$

because each field element has characteristic two. Every checked block pair was selected from $\mathcal{V}$, so Algorithm 6 accepts. $\square$

Attack algorithm. The attacker enumerates $\mathcal{V}$ from public parameters, fills all n/ℓ blocks of (e_1, e_2) with valid pairs, chooses σ , computes the two public matrix products, and outputs Equation (2).

The choice $\sigma = 0$ reduces the attack to $z_i = e_i$ and shows that the acceptance identity is independent of the key. A nonzero σ variant exercises the complete public-matrix path and demonstrates that excluding the zero signature would not help. The result holds for every chosen σ .

Freshness and attack class. The raw tuple z is computed for the attack and never submitted to a signing oracle; there is no signing query at all. Thus the output is a fresh message/signature pair for the literal message space. Because the attacker chooses the signature and obtains a matching message rather than targeting a preselected external document, the precise class is a zero-query existential forgery. That class is sufficient to violate existential unforgeability of the raw-message construction.

4 Primary implementation

The primary standard-library implementation represents binary row vectors as Python integers. It reconstructs all relevant paper algorithms rather than hard-coding an accepted transcript:

1. It constructs the Equation (1) staircase generator G for the full PS1 and PS2 dimensions.
2. It samples G_1 as a full binary matrix and defines $G_2 = G + G_1$.
3. It applies an invertible scrambler, a common outer block permutation, and independent inner permutations to form both public matrices.
4. It exhaustively derives $\mathcal{V}$ from Algorithm 3’s universal permutation condition.
5. A public-only function produces zero- and nonzero- σ forgeries.
6. The ordinary Algorithm 6 function checks every block.
7. Invalid-block and strict representation controls test rejection.

The deterministic seeds make the public-package output reproducible and are not used by the attack. The forged-message digests and public-key digests are stable. The private staircase matrix, split, scrambler, and permutations are held separately and are never passed to the forger.

4.1 Primary results

The sealed primary run produced the following measurements:

Row	variant	blocks	$ \mathcal{V} $	attack	verify	result
PS1	$\sigma = 0$	714	24	1.867 ms	0.420 ms	accept
PS1	nonzero σ	714	24	0.469 ms	0.504 ms	accept
PS2	$\sigma = 0$	561	40	0.423 ms	0.460 ms	accept
PS2	nonzero σ	561	40	0.501 ms	0.523 ms	accept

All four variants cover the complete valid-split set for their row. The ordinary verifier checked all 2,550 positive blocks. Four one-invalid-block controls and sixteen format controls rejected. The complete run, including both key generations and all controls, took 3.21 seconds wall-clock and used 20,992 KiB peak resident memory. The maximum attack-only measurement was 1.867 milliseconds.

5 Independent reproduction

The sealed independent reproduction was written separately from the primary implementation. It used OS-random full-row keys and a dense random full-rank scrambler. Key generation, the attacker, and verification ran as separate processes. The key generator persisted no private material, and the attacker process received only a serialized public key and an output path; it exposed no signing-oracle interface.

For one fresh key at each full row, the independent verifier accepted both a zero- σ and a nonzero- σ forgery. It rejected an invalid block, a changed raw message, a changed signature, and four representation controls at each row. Peak resident memory was 20,480 KiB, and each of the six stages took at most 0.66 seconds. The sealed manifest has SHA-256 digest

c0e2c712fccb11803285f2f0bf03c16cee226b7b15f91d0eaab9ece71fae064d.

The package also contains a deterministic, separately structured companion reproducer. It uses a dense full-rank scrambler and independent matrix, permutation, enumeration, attack, and verification routines. It repeats both variants and the controls at both rows, yielding stable reference output.

6 Controls and interpretation

6.1 Invalid-block controls

At both rows, $(0, 0)$ is absent from $\mathcal{V}$. The controls replace one block of z so that Algorithm 6 recovers exactly this pair. The primary zero variants fail at the first block; the nonzero variants place the bad block at the end and therefore exercise the complete rejection scan. The independent implementation also changes a valid signature by one bit, selected by an exhaustive search, while keeping z fixed. Verification rejects.

6.2 Representation controls

The mathematical algorithms declare fixed-dimensional vector spaces. The implementations enforce those dimensions explicitly. Wrong message arity, wrong component width, wrong signature width, non-hexadecimal encodings in the sealed independent run, and values outside the declared vector spaces all reject before the algebraic predicate. These tests rule out an accidental always-accepting verifier and separate the identity from encoding behavior.

6.3 Separate signer-correctness observation

The primary implementation also records a correctness gap that the forgery does not use. Algorithm 5's stated decoder contract ensures only that an aggregate decoded error belongs to E_ℓ . Algorithm 6 demands the stronger condition that each component tuple belongs to $\mathcal{V}$, which quantifies over every independent pair of inner permutations.

Take two distinct unit vectors $a, b \in \mathbb{F}_2^\ell$. Their sum has Hamming weight two and belongs to the printed E_ℓ . Independent permutations can align the two units, however, making their sum zero, which is outside E_ℓ . Thus $(a, b) \notin \mathcal{V}$. The primary run constructs key-consistent full-row transcripts that meet the stated aggregate decoder condition at all 714 PS1 blocks or all 561 PS2 blocks, yet Algorithm 6 rejects every block. This is separate from the attack, which directly constructs accepting transcripts.

7 Limitations and repair direction

The result is exact for the mathematical raw- z interface in the pinned paper. It does not establish an attack on an absent external transform from meaningful documents to z . In particular, if a deployment independently specified a preimage-resistant map from documents into the complete z space, the attacker would additionally have to find a document mapping to the constructed tuple. That is a different problem and no such transform appears in the target.

The experiment does not estimate a success probability. Once public valid errors are selected, acceptance is an algebraic identity for every key. The fixed seeds in the package affect only reproducibility of test keys.

Banning $\sigma = 0$ is insufficient because Equation (2) works for every nonzero σ . A repair must define the signed message domain and prevent an adversary from solving backward from a freely chosen signature to an accepted signed value. An explicit, analyzed preimage-resistant encoding from external messages into z could block this particular route, but it would be a new transform requiring its own correctness and security proof. More generally, the verification relation should bind the accepted object to the signing key under a standard unforgeability reduction rather than expose a publicly solvable affine equation.

8 Conclusion

For both printed full parameter rows, a zero-query adversary can choose public valid error blocks and any signature vector, then solve the verifier equation for a fresh raw z tuple. Algorithm 6 cancels the duplicated public codeword and accepts deterministically. Two independently structured implementations reproduce the attack, and the measured construction takes at most 1.867 milliseconds in the sealed primary run. The exact claim is a public-key-only existential forgery against the raw tuple specified by Algorithms 5–6. It does not extend to arbitrary documents or an unspecified external hash wrapper.

Primary, independent, history, intended-scope, amended adversarial, and independent publication reviews all passed. The result is classified `VERIFIED_ATTACK`.

Acknowledgements

The computations were performed on the Norwegian Research and Education Cloud (NREC), using resources provided by the University of Bergen and the University of Oslo.

References

- [1] D. Gligoroski and S. Samardjiska. A Digital Signature Scheme Based on Random Split of St-Gen Codes. IACR Cryptology ePrint Archive, Report 2016/391, 2016. <https://eprint.iacr.org/2016/391>.
- [2] D. Gligoroski, S. Samardjiska, H. Jacobsen, and S. Bezzateev. McEliece in the world of Escher. IACR Cryptology ePrint Archive, Report 2014/360, 2014. <https://eprint.iacr.org/2014/360>.
- [3] S. S. S. Shehata. *Post Quantum Cryptography with random split of St-Gen codes*. Master’s thesis, Norwegian University of Science and Technology, June 2017. Supervised by Danilo Gligoroski. <https://hdl.handle.net/11250/2450584>.
- [4] D. Moody and R. Perlner. Vulnerabilities of “McEliece in the World of Escher.” In *Post-Quantum Cryptography 2016*, LNCS 9606, pp. 104–117. Springer, 2016. https://doi.org/10.1007/978-3-319-29360-8_8.
- [5] S. Goldwasser, S. Micali, and R. L. Rivest. A digital signature scheme secure against adaptive chosen-message attacks. *SIAM Journal on Computing*, 17(2):281–308, 1988. <https://doi.org/10.1137/0217017>.
- [6] A. J. Menezes, P. C. van Oorschot, and S. A. Vanstone. *Handbook of Applied Cryptography*, Chapter 11. CRC Press, 1996. <https://cacr.uwaterloo.ca/hac/about/chap11.pdf>.